

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES DISTRIBUIDAS [20701]

NIVEL / PARALELO: 7MO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 29-04-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: FCCDD-LAB-TIC-201

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Resolución de problemas prácticos

N. HORAS: 9

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Implementa soluciones de computación paralela y distribuida empleando tecnologías de procesamiento concurrente y plataformas de computación en la nube, evaluando su rendimiento e impacto.

Tema de la práctica

Configuración de un clúster de base de datos distribuida de tres nodos con CockroachDB en contenedores Docker, implementación de la estrategia de fragmentación y replicación del PFC, verificación de tolerancia a fallos (Raft consensus), y análisis comparativo de rendimiento distribuido vs. centralizado.

3 OBJETIVO

Objetivo General:

Implementar, configurar y verificar un clúster de base de datos distribuida funcional de tres nodos, demostrando transparencia de fragmentación y replicación, comportamiento ante fallos de nodos y recuperación automática.

Objetivos Específicos:

1. Configurar un clúster CockroachDB de tres nodos en contenedores Docker con replicación automática.
2. Implementar el esquema de datos del PFC con la estrategia de fragmentación diseñada en la Semana 10.
3. Demostrar tolerancia a fallos: detener un nodo y verificar que el sistema continúa operando (quórum Raft).
4. Medir y comparar rendimiento de consultas distribuidas vs. nodo único.

4 FUNDAMENTO TEÓRICO

Desarrollar en LaTeX el fundamento teórico COMPLETO de la Unidad 2. Las definiciones formales de fragmentación DEBEN incluir notación de álgebra relacional con entornos matemáticos LaTeX. Cada subtema: mínimo 300 palabras, ≥2 referencias IEEE con DOI.

3.1 Fundamentos y Arquitecturas de Bases de Datos Distribuidas

Desarrollar: definición formal de BDD, los 12 objetivos de diseño de Date, arquitecturas de SGBD distribuidos (cliente-servidor, colaborativa, middleware), ocho formas de transparencia en BDD, y diferencia entre homogeneidad y heterogeneidad.

Referencias bibliográficas mínimas [IEEE]:

- [1] M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed. Cham, Switzerland: Springer, 2020. doi: 10.1007/978-3-030-26253-2.
- [2] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017. ISBN: 978-1-4493-7332-0.

3.2 Fragmentación y Replicación de Datos

Desarrollar: definición formal con condiciones de corrección (completitud, reconstrucción, disjunción), fragmentación horizontal con álgebra relacional, fragmentación vertical con análisis de afinidad, fragmentación mixta, replicación síncrona vs. asíncrona con análisis de trade-offs.

Referencias bibliográficas mínimas [IEEE]:

- [3] M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed., ch. 3-4, pp. 79-180.
- [4] M. Kleppmann, Designing Data-Intensive Applications, ch. 5-6, pp. 151-260.

3.3 Control de Concurrencia Distribuida

Desarrollar: problema de concurrencia en BDD, protocolo 2PL (bloqueo en dos fases) distribuido y estricto, detección de deadlock distribuido con grafos wait-for, protocolo de commit en dos fases (2PC) con análisis de fallos, protocolo 3PC.

Referencias bibliográficas mínimas [IEEE]:

- [5] M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed., ch. 7-8, pp. 261-360.
- [6] M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed. Maastricht, The Netherlands: Maarten van Steen, 2023, ch. 6.

3.4 Consistencia, Disponibilidad, PACELC y Sistemas NoSQL

Desarrollar: teorema CAP y su interpretación moderna, modelo PACELC de Abadi (2012), consistencia eventual vs. fuerte, modelos de CockroachDB (serializable), modelos de datos NoSQL (documentos, clave-valor, columnar, grafos), análisis comparativo CockroachDB vs. MongoDB vs. Cassandra.

Referencias bibliográficas mínimas [IEEE]:

- [7] E. Brewer, 'CAP twelve years later,' IEEE Comput., vol. 45, no. 2, pp. 23-29, Feb. 2012. doi: 10.1109/MC.2012.37.
- [8] D. J. Abadi, 'Consistency tradeoffs in modern distributed database system design,' IEEE Comput., vol. 45, no. 2, pp. 37-42, Feb. 2012. doi: 10.1109/MC.2012.33.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- TeX Live 2023+ o Overleaf con paquetes completos
- DB Beaver o TablePlus como cliente SQL visual (opcional)
- Python 3.10+ con: psycpg2-binary, faker, pandas, matplotlib
- CockroachDB v23.1+ (imagen: cockroachdb/cockroach)
- Docker Desktop 4.x con Docker Compose Plugin

Equipos

- Computadora con ≥16 GB RAM (necesario para 3 nodos CockroachDB simultáneos,...)

Insumos y reactivos

- 40 GB de espacio libre en disco para volúmenes Docker.

6 PROCEDIMIENTOS

Paso 1 Levantamiento del Clúster CockroachDB

- Crear docker-compose.yml con tres nodos: crdb-node1 (puerto 26257/8080), crdb-node2 (26258/8081), crdb-node3 (26259/8082). Red interna: roach-net (bridge).
- Inicializar clúster: `docker exec -it crdb-node1 ./cockroach init --insecure``
- Verificar estado: `docker exec -it crdb-node1 ./cockroach node status --insecure`` — todos los nodos con `is_live=true`.
- Captura del dashboard CockroachDB (<http://localhost:8080>) con los 3 nodos activos.

⚠ **Nota:** Si hay problemas de memoria, limitar a 512m por nodo en docker-compose con `mem_limit`.

Paso 2 Implementación del Esquema del PFC

- Conectar: ``docker exec -it crdb-node1 ./cockroach sql --insecure``
- Crear base de datos y tablas del PFC (mínimo 3 tablas con relaciones).
- Insertar $\geq 10,000$ registros con script Python (psycopg2 — compatible con CockroachDB).
- Verificar distribución de rangos: ``SHOW RANGES FROM TABLE tabla_principal;``

Paso 3 Verificación de Tolerancia a Fallos

- Ejecutar consulta: ``SELECT COUNT(*) FROM tabla_principal;`` — registrar resultado y tiempo.
- Detener nodo 2: ``docker stop crdb-node2``
- Ejecutar la misma consulta inmediatamente — debe responder correctamente (quórum Raft: 2 de 3 nodos).
- Esperar 30 seg. Ejecutar `SHOW RANGES` — documentar redistribución de rangos.
- Reiniciar nodo 2: ``docker start crdb-node2`` — documentar tiempo de reintegración.

⚠ **Nota:** Grabar un video de pantalla durante la prueba de tolerancia a fallos (entregar con el repositorio).

Paso 4 Medición de Rendimiento

- Ejecutar 5 consultas SQL del PFC en el clúster (3 nodos) con `EXPLAIN ANALYZE`.
- Ejecutar las mismas consultas en una instancia única: ``cockroach start-single-node --insecure``.
- Tabla comparativa: Consulta | T. clúster (ms) | T. nodo único (ms) | Factor de mejora.
- Analizar: ¿en qué consultas el clúster es más rápido? ¿en cuáles más lento? ¿por qué?

Paso 5 Documento LaTeX

- Incluir: fundamento teórico U2 completo, configuración Docker en Istlisting YAML, SQL en Istlisting, capturas del dashboard como figuras, tabla booktabs de rendimiento, análisis de prueba de fallos con diagrama de secuencia del comportamiento Raft, conclusiones y bibliografía (≥ 6 referencias IEEE).
- Mínimo 12 páginas de contenido. Compilar sin errores.

7 BIBLIOGRAFÍA

- CockroachDB Inc., 'CockroachDB technical documentation v23.2,' Cockroach Labs, 2024. [Online]. Available: <https://www.cockroachlabs.com/docs/>
- M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed. Maastricht, The Netherlands: Maarten van Steen, 2023.
- D. J. Abadi, 'Consistency tradeoffs in modern distributed database system design,' IEEE Comput., vol. 45, no. 2, pp. 37-42, Feb. 2012. doi: 10.1109/MC.2012.33.
- E. Brewer, 'CAP twelve years later,' IEEE Comput., vol. 45, no. 2, pp. 23-29, Feb. 2012. doi: 10.1109/MC.2012.37.
- M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017. ISBN: 978-1-4493-7332-0.
- M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed. Cham, Switzerland: Springer, 2020. doi: 10.1007/978-3-030-26253-2.

8 ANEXOS

- CockroachDB usa el algoritmo Raft. Explica cómo garantiza consistencia cuando un nodo falla, con referencia a los conceptos de quórum, líder y log de transacciones.
- Compara el modelo de consistencia de CockroachDB (Serializable) con Apache Cassandra (eventual consistency). ¿Cuál elegirías para el PFC y por qué?
- ¿Cómo implementarías fragmentación horizontal explícita con `PARTITION BY RANGE` en CockroachDB? Diseña la estrategia para la tabla principal del PFC.
- Si el PFC requiere transacciones multi-tabla en diferentes nodos, ¿cómo garantiza CockroachDB la atomicidad? Explica el commit de dos fases interno.

9 RESULTADOS DE ESTUDIANTES

GUERRERO ULLOA GLEISTON CICERON
CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
CI: 1205316456
COORDINADOR DE CARRERA

